

嵌入式 Linux 系统

嵌入式系统的定义:

以应用为中心, 以计算机技术为基础, 软硬件可裁剪, 对功能、可靠性、成本、体积、功耗有严格约束的一种专用计算机系统
非PC 智能电子设备

嵌入式系统的应用:

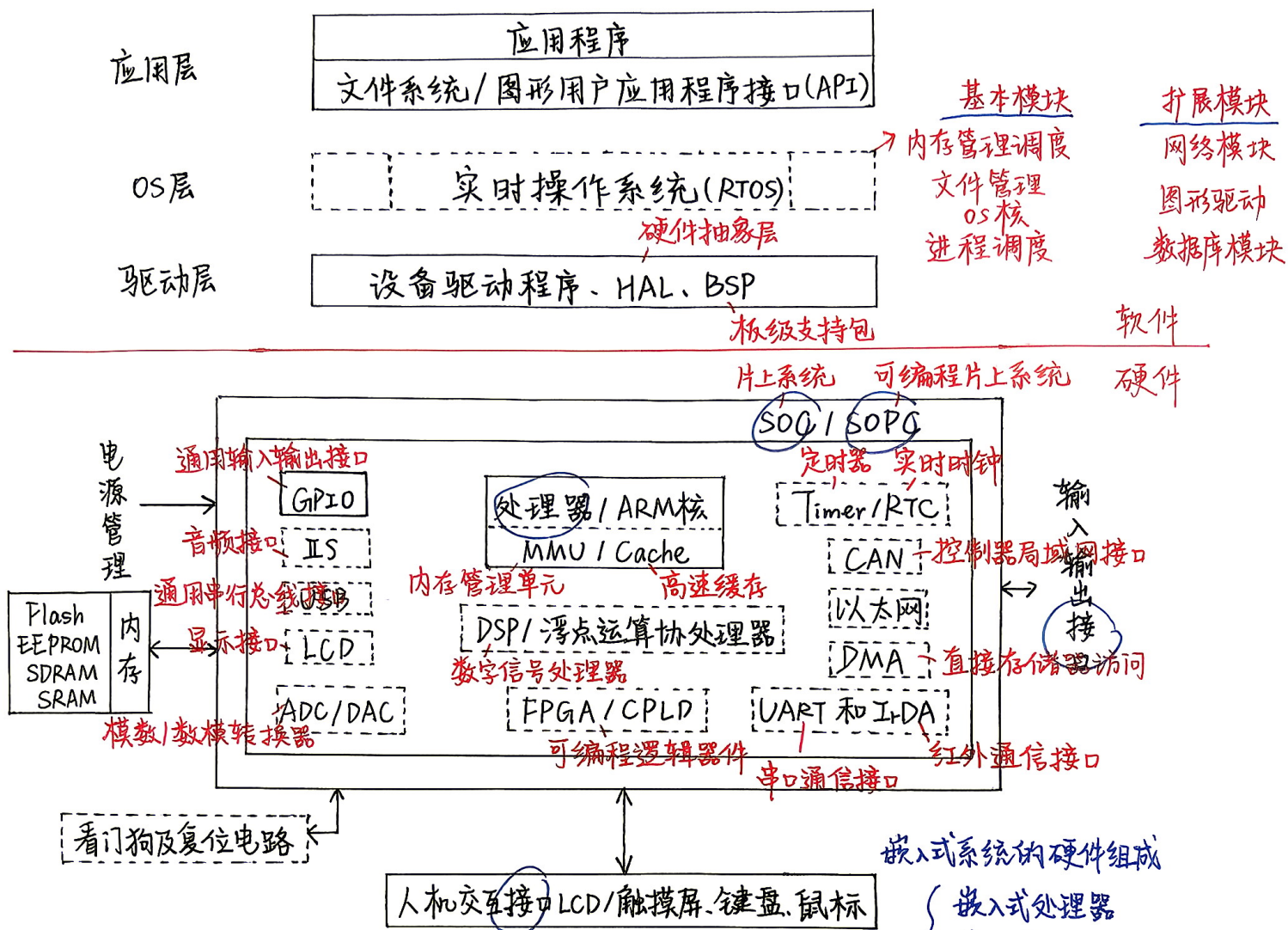
工业	军事国防	消费电子	网络
工控设备	军事电子	信息家电	网络设备
智能仪表	⋮	智能玩具	电子商务
汽车电子		通信设备	⋮
⋮		移动终端	⋮

从无线电、电话、计算机、航海仪器、日用电器等的发展, 可以看出其发展的规律大致是 从模拟到数字、从单一功能到多功能集成、从无处理器到多处理器、从孤立设备到网络互联、从大型设备到小型化、便携化

无线连接是“数字胶水”, 无线通信把各种嵌入式设备连接起来, 共同组成网络化、智能化的数字世界

另外, 注意实时性, 实时操作系统(RTOS)

嵌入式系统的组成:



典型嵌入式处理器:

① MIPS 处理器

无内部互锁流水级的微处理器, 基于RISC架构

② PowerPC 处理器

可伸缩性好, 灵活性强, 基于RISC架构

③ Sparc 处理器 (Sun 公司)

可扩展, 基于RISC架构

④ ARM 处理器

嵌入式领域应用最广泛的RISC处理器, 低功耗、低成本、小体积、高性能,

ARM公司主要通过IP授权的方式推广其处理器架构。

嵌入式系统的硬件组成

- 嵌入式处理器
- 嵌入式SoC
- 可编程片上系统SOPC
- 嵌入式外围接口电路和设备接口

软件层次结构

- 应用程序
- 操作系统的应用程序接口(API)
- 实时操作系统(RTOS)
- 驱动层程序

外围设备：

① 存储器

存储数据和程序的功能部件，分为

- 静态易失型存储器 (RAM、SRAM)
- 动态存储器 (DRAM) (电可擦除可编程只读存储器)
- 非易失性存储器 (MASK ROM、EPROM、EEPROM、FLASH)
- 硬盘、软盘、CD-ROM (可擦除可编程只读存储器)
常用紫外线

② 通信接口

应用最广泛的接口设备包括

- RS-232 接口 (串口 UART)
- USB 接口 (通用串行总线接口)
- IrDA (红外线接口)
- SPI (串行外围设备接口)
- I2C
- CAN 总线接口
- Bluetooth 接口
- Ethernet 接口
- IEEE 1394 接口
- 通用可编程接口 GPIO

③ 输入输出设备

例如 CRT、LCD、触摸屏等，触摸屏可以方便地实现鼠标和键盘功能

④ 设备扩展接口

简单的嵌入式系统若具有简单记事本、备忘录及日程计划等功能的 PDA (个人数字助理)，其所需存储的数据量不需要很大内存。

由于目前嵌入式系统越来越复杂，所需内存容量越来越大，使系统的体积和成本都增大，很多高端的嵌入式系统都预留了可扩展存储设备接口，以便用户日后购买符合扩展接口规格的装置直接接入系统使用。

常用的扩展卡如 CF 卡、SD 卡、Memory Stick 等

⑤ 电源及辅助设备

嵌入式系统力求小型化、重量轻以及电源使用寿命长，很多体积过大或重量过重的机型 (不便携) 已被淘汰

嵌入式操作系统

Operating System

计算机由硬件、软件两部分组成，操作系统是配置在计算机硬件上的第一层软件。操作系统可以理解为弥补硬件差异的界面（隐藏硬件），让应用程序可以在上面运行。

通过由操作系统统一提供的系统界面来写应用程序，无须考虑不同硬件的差异，程序设计人员可以专注于擅长领域的开发。

嵌入式系统一般具有实时的特点，一个优先级高的任务可以获得立即的、没有延迟的服务，它无需等待任何其他服务，在得到CPU的使用权后可以一直执行到工作结束或有更高等级的进程出现为止。

嵌入式操作系统就是支持嵌入式系统工作的操作系统，其大大提高了是嵌入式系统的灵魂

嵌入式系统开发的效率，减少了系统开发的总工作量，提高了嵌入式应用程序的可移植性。

从用户的观点看，OS是用户与计算机硬件系统间的接口，在OS的帮助下可以方便快捷、安全可靠地操作计算机硬件与运行自己的程序。

从资源管理的观点看，OS是计算机系统资源的管理者。

实时操作系统可依据实际应用环境的要求对内核进行剪裁与重新配置。

有些组成部分会因实际应用不同而不同，但实时内核、网络组件、文件系统、图形接口等重要组成部分是基本不变的。

Real-Time

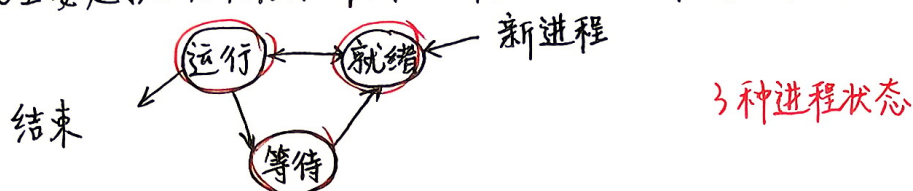
大多嵌入式系统应用在实时环境中，因此嵌入式操作系统与实时操作系统密切联系在一起。

嵌入式操作系统相对于一般操作系统而言,仅指操作系统的内核(或微内核),其他的诸如窗口系统界面或通讯协议等模块,可另外选择。

目前大多数的嵌入式操作系统必须提供以下管理功能:

① 多任务管理

所有嵌入式操作系统都是多任务的,所谓多任务,大都指多线程方式或多进程方式。操作系统主要是提供调度机制来控制这些执行程序的起始、执行、暂停、结束



② 存储管理

由于系统资源非常有限,一般不用虚拟内存管理而是用动态内存管理方式。当程序的某一部分需要使用内存时,利用操作系统提供的分配函数来处理,一旦使用完毕就通过释放函数来释放所占用的内存,使得内存可重复使用。

③ 周边资源管理

除系统本身的中央处理器、内存之外,还有许多不同的周边系统,例如输入/输出设备、通讯端口、外接控制器等,因此必须提供周边资源的驱动程序以便资源管理与应用程序使用。(应用程序必须向OS注册请求机制,然后等待OS安排资源)

④ 中断管理

由于查询方式需占用大量CPU时间,因此大多采用中断方式处理外部事件与I/O请求。中断管理负责中断的初始化安装、现场保存与恢复、中断栈的嵌套管理等。

典型的嵌入式操作系统

嵌入式系统所需操作系统特征:

高度简练、界面友善、质量可靠、应用广泛、易开发、多任务、价格低廉

- 商用型: VxWorks、Windows CE、Psos、Palm OS、OS-9、Lynx OS、QNX、LYNX ...
- 免费型: Linux、 $\mu C/OS-II$

嵌入式软件开发

宿主机 - 目标机 交叉开发

在PC机上写 $\rightarrow$ 编译 $\rightarrow$ 下载 $\rightarrow$ 调试程序, 程序最终在 ARM、单片机、DSP、

特点:

嵌入式等目标硬件上运行

软硬件必须协同设计, 需要软硬件开发环境和工具, 需要新的任务设计方法,

开发完成后, 系统应用程序代码需固化到系统中进行功能、性能、可靠性测试

挑战:

软件要求更高, 开发工作量与难度更大

开发环境:

① 交叉开发环境

交叉开发软件一般是一个整合编辑、编译汇编链接、调试、工程管理及函数库等功能模块的集成开发环境(IDE)

断点设置、单步执行、寄存器与内存查看...

嵌入式交叉开发环境的宿主机到目标机的3种调试通道:

- 在线调试 / 在线仿真: 电脑通过调试接口控制跑在芯片上的程序
- 例如: 基于 JTAG 的 ICD、背景调试模式 (BDM)
- OCW (On-Chip Debugging) On-Chip Emulator
- 在线仿真器 ICE
- ROM 监控器
- In-Circuit Debugger

② 软件模拟环境 (亦称指令集模拟器, ISS)

仅作为初步调试, 用作用户程序的模拟运行, 检查语法、程序结构等简单错误, 无法完全代替真正的硬件环境, 用户最终仍须在真实硬件环境中运行调试.

③ 评估电路板

是应用目标板出来前的软件测试、硬件调试电路板, 即开发者使用的学习板、实验板

宿主机与目标机

宿主机 (Host) 是开发人员使用的 PC, 负责程序编辑、交叉编译、链接、下载、调试控制,

目标机 (Target) 是嵌入式开发板或目标硬件, 负责运行生成的目标程序.

二者通过 JTAG、串口、USB、以太网、SWD 等连接.

编译与运行的 CPU 架构不一致

所谓“交叉开发环境”是指开发工具运行在宿主机上, 但生成的程序运行在另一目标处理器/目标板上, 所谓“交叉”就是因为编译平台 ≠ 运行平台

比如在 Windows 电脑上用 IDE 写 ARM 程序, 编译后生成的是 ARM 芯片可执行的机器码, 而不是电脑 CPU 可直接执行的程序.

一个完整的嵌入式开发环境通常包括什么? 连接 PC 与目标板, 支持断点、单步、内存与寄存器查看

宿主机、目标机、集成开发环境、交叉编译工具链、调试器/仿真器、下载烧写工具、

↓
写代码、管理工程、
编译、调试

↓
例如 Flash 编程器

在 x86 架构宿主机上编译生成 ARM/DSP 等不同架构芯片专用的二进制程序

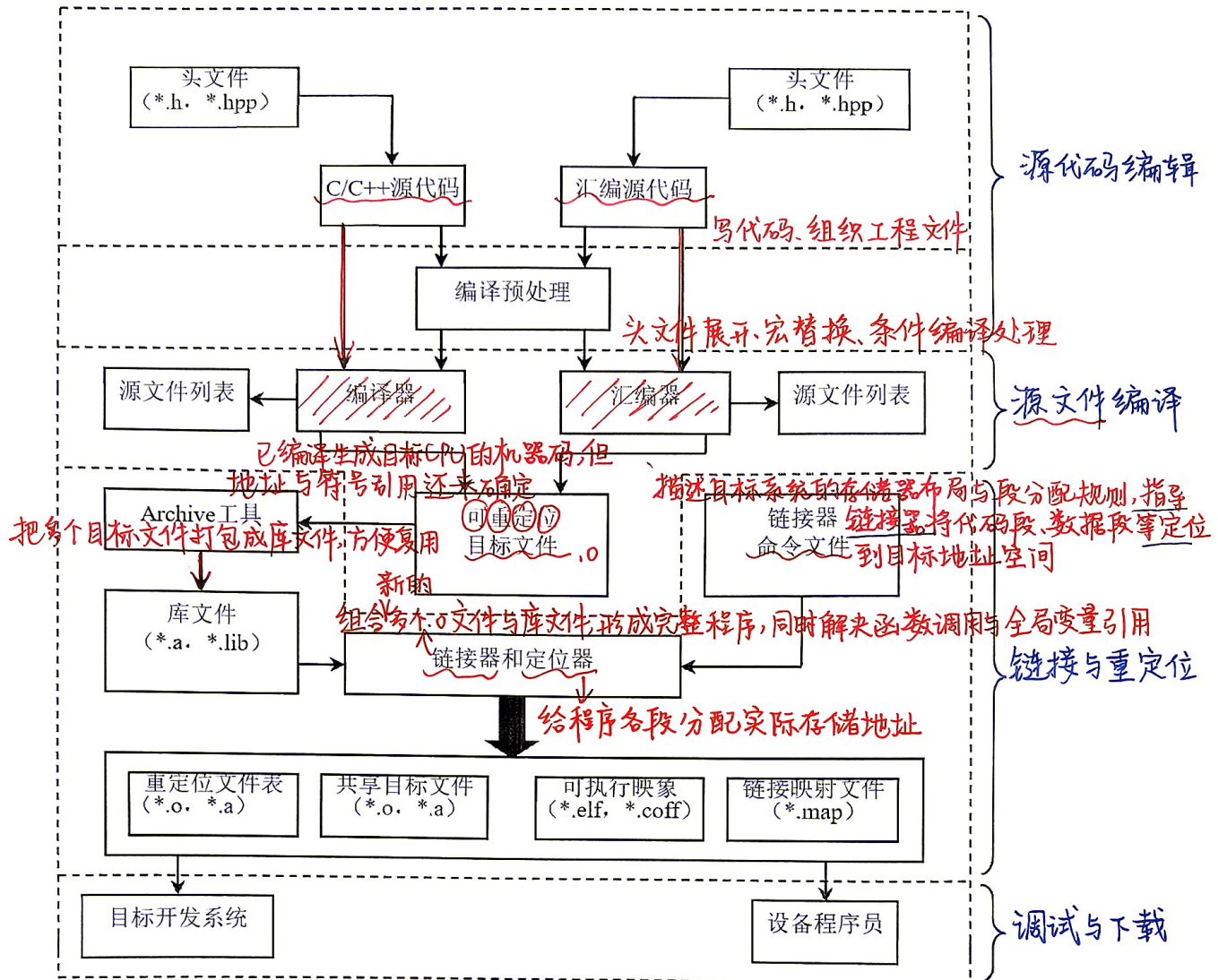
包括交叉编译器、汇编器、链接器等, 将程序变为目标芯片可执行的代码

评估板、目标开发板、各种连接线、电源适配器

↓
实验、测试、调试用

↓
真正运行程序的硬件平台

嵌入式软件开发流程图



嵌入式应用软件开发一般包括源代码编辑、预处理、编译、汇编生成目标文件、链接、定位、生成可执行文件、下载烧写至目标系统、调试、运行等步骤:

- ① 在宿主机上编写 C/C++ 或汇编源程序并进行预处理
- ② 由交叉编译器和汇编器生成可重定位目标文件
- ③ 由链接器将目标文件与库文件进行链接, 并根据链接器命令文件完成地址定位, 生成可执行文件
- ④ 由烧写工具下载烧录至目标开发板, 由调试器在线调试, 无误后程序可在目标系统中独立运行

基于交叉开发环境的嵌入式应用软件开发的5个基本阶段:

开发环境^①建立 → 源代码^②编辑 → 交叉编译和链接^③ → 重定位和下载^④ → 联机调试^⑤

在确保软件的正确性、实时性的前提下, 必须关注软件的可移植性与可重用性

嵌入式软件与通用软件的不同在于嵌入式应用软件高度依赖于目标应用的软硬件环境, 软件的部分任务功能函数由与处理器密切相关的汇编语言完成, 可移植性差

一个运行良好的嵌入式软件或其中部分子程序可能未来被应用于类似领域.

在已被反复应用和维护、具有好的稳定性的原有代码上进行移植会

减少开发周期、提高开发效率、节约开发成本.

如何提高应用软件的可移植性与可重用性?

① 少用或不用汇编语言, 多用高级语言

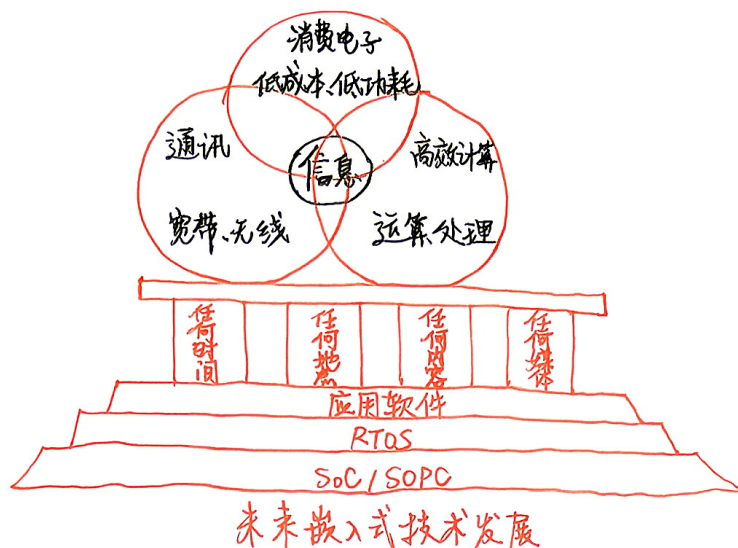
② 将不可移植部分局域化

③ 提高代码的可重用性

嵌入式系统开发

考虑因素:

功能 可靠实用、便于升级
实时 并发处理、及时响应
体积 符合要求、结构紧凑
接口 符合规范、易于操作
配置 精简稳定、维护便利
功耗 管理严格、成本低廉



软硬件协同设计 (该方法的特点是在设计时,从系统功能实现的角度,把软硬件同时考虑进去)

嵌入式系统设计是使用一组物理硬件和软件来完成所需功能的过程。

系统是由硬件、软件或二者结合构成的能完成特定功能的整体。

由于嵌入式系统是专用的系统,软硬件设计紧密结合、相互协调,即.在设计时,从系统功能、性能、功耗、成本、开发周期等需求出发,

同时考虑软硬件的实现方式,对系统功能进行合理划分。

常将灵活性强、调整频繁的功能交由软件实现,

将实时性强、运算量大或需要高速处理的功能交由硬件实现

可以最大限度地利用有限资源、缩短开发周期,且能取得更好的设计效果。

基本流程:

系统定义与需求分析 → 设计方案初步确立 → 方案评审论证与性价比评估 →

完善初步方案并实施 → 软硬件集成测试 → 系统功能性能测试及可靠性测试

发展趋势: 智能化、网络化、高性能、低功耗、小型化、安全化、个性化。 (高效)

在“以ARM为核心的32位浪潮”之后,至今32位ARM仍很重要,但高端嵌入式已大量进入

64位ARM、GPU、NPU、DSP、异构SoC、AI加速器、RISC-V阶段。

WiFi、Bluetooth、蜂窝网络、以太网、LoRA、NB-IoT等方式接入

嵌入式系统与Internet融合,使嵌入式设备具备联网通信、远程监控、数据共享、在线升级、智能服务的能力,推动了物联网的发展。(注意网络安全与隐私保护)

3C (Computer-计算、Communication-通信、Consumer-消费电子) 技术融合,可随时随地获取、处理、发布信息。